Codex 실전 운용 가이드

터미널·병렬 작업·검증·제출을 연결하는 사전 준비서

SK하이닉스 AI 해커톤 2026 준비용 · 2026-10-31 검토

이 자료의 목표는 Codex에게 많은 일을 시키는 데 그치지 않고, 핵심 기능이 실제로 동작하는지 확인하고 자신의 판단을 설명할 수 있도록 돕는 것이다.

공식 승인 자료가 아니다. 실제 과제 README, 제공 환경, 최신 공지가 우선이다. 로컬에서 확인한 Codex CLI는 0.166.0이며 응시 환경의 버전·메뉴·권한은 다를 수 있다.

읽는 순서

처음 준비할 때: 1~3장 → 6장 → 10~12장.

여러 터미널·에이전트를 쓸 때: 4~7장.

작업이 막히거나 제출 직전일 때: 8~11장.

구성

1. 규정과 작업 경계
2. 시작 전 15분 점검
3. 첫 요청과 핵심 흐름
4. 세 개의 병렬 개념
5. Bash 병렬 실행과 종료 코드
6. 서브에이전트 위임
7. 파일 충돌과 통합
8. 대기·오류·자원 문제
9. 컨텍스트와 인수인계
10. 실제 검증과 증거
11. 시간 배분과 제출
12. 바로 쓰는 요청문과 출처

1. 규정과 작업 경계

공개 준비 저장소를 미리 만들어 제공 환경에서 가져오는 흐름과, 시험 중 생성한 파일을 외부로 보내는 행위는 구분해야 한다. 이 저장소는 전자를 위한 것이다.

상황	적용할 원칙
사전 준비	범용 지침·스킬·템플릿을 공개 저장소에 준비
실제 과제	제공된 Codex와 작업 공간에서 구현·검증·기록
공개 자료 가져오기	안내가 허용하는 Git Clone·다운로드 경로 사용
외부 전송	문제·파일·작업 결과물 전송 및 Git push 금지
자동 조작	Computer Use 금지; UI는 사람이 플랫폼 Browser에서 확인
후속 서술형	AI·복사 붙여넣기 없이 직접 답변

근거: 제공된 브리프 1쪽, 상세 가이드 2~5쪽. 공개 상태 유지 조건은 사용자가 전달한 추가 공지에 근거한다. 원본 PDF는 재배포하지 않았다.

모델과 도구

개인 API 키·개인 유료 구독·별도 외부 AI 연결을 사용하지 않는다. Astra 제한을 포함한 실제 허용 모델은 응시 화면과 안내에서 확인한다. 최신 공개 모델을 임의로 지정하거나 로컬 설정 파일을 실전 환경에 덮어쓰지 않는다.

서브에이전트와 여러 터미널이 있다는 사실이 무제한 동시 사용을 보장하지 않는다. 처음에는 필요한 작업만 나누고 CPU/RAM·응답 시간을 보며 조절한다. 운영 설정과 감독·연결 프로세스는 수정하지 않는다.

준비 전략과 공식 조건을 구분

이 문서의 역할 분담, 시간표, 요구사항 ID, 기록 양식은 준비 전략이다. 제출 파일명·배점·과제 유형·권장 동시 에이전트 수를 공식 조건으로 확정하지 않는다.

2. 시작 전 15분 점검

사전 체험은 제공 안내상 1회·15분이다. 연습 가능 시간과 실제 화면 안내를 먼저 확인한다. 감독 절차는 별도 Proctor 가이드를 따른다.

시간 예시	확인할 것	성공의 증거
0~3분	README·파일 저장·터미널	만든 작은 파일을 다시 읽을 수 있음
3~6분	Codex 실행과 짧은 요청	응답, 승인 대기 상태를 식별
6~10분	제공 프로젝트 서버	실제 포트와 Browser 연결 확인
10~12분	핵심 입력·버튼	사람이 결과 변경을 직접 확인
12~15분	종료 흐름·예시 질문	화면 안내와 직접 답변 방식 숙지

위 배분은 연습안이다. 감독·접속 점검에 필요한 시간은 실제 상황에 맞춰 조정한다.

짧은 확인 명령

```
pwd
ls
node --version
python3 --version
codex --version
```

도구가 없으면 오류 자체가 환경 정보다. 모든 도구를 설치하려고 하지 않는다. 실제 과제에 필요한 것만 확인한다.

Codex 입력창에서 `/status`, `/model`, `/` 메뉴를 확인한다. Bash 프롬프트에 슬래시 명령을 입력하는 것과 Codex 입력창에서 사용하는 것은 다르다. 특정 메뉴가 없으면 제공 버전의 기능으로 진행한다. 명령 기능의 일반 근거: [OpenAI 개발자 명령 문서].

피할 시작 실수

개인 환경의 인증·설정·플러그인을 통째로 이식하지 않는다. 준비 저장소 전체를 과제 코드로 오인하지 않는다. 실제 제출 폴더를 먼저 알아야 서버·소스·문서가 흩어지지 않는다.

3. 첫 요청과 핵심 흐름

첫 요청에는 목표, 근거 파일, 제약, 완료를 확인할 방법을 넣는다. 한 번의 요청에 모든 구현을 세세하게 고정하기보다 필수조건을 확인하고 가장 중요한 경로부터 만든다.

> 과제 README와 기존 AGENTS.md, 제공 데이터를 읽어줘. 필수 기능·제출물·성공 기준을 요구사항 ID로 정리해줘. 사실과 가정을 구분하고, 가장 중요한 사용자 흐름 하나를 입력부터 결과까지 구현해줘. 기존 스택을 우선하고 검증 결과를 남겨줘.

요구사항을 행동으로 바꾸기

모호한 표현	확인 가능한 표현 예시
대시보드 구현	지역 필터를 바꾸면 해당 집계와 표가 함께 갱신
데이터 처리	빈 값·중복 처리 기준을 정하고 작은 입력으로 검산
사용성 향상	잘못된 입력에 수정 방법이 표시되고 재시도 가능
저장 지원	새로고침 또는 재시작 중 요구된 범위에서 데이터 유지

예시는 출제 예상이 아니다. 실제 과제의 사용자와 업무 기준에 맞게 바꾼다.

기술 선택

제공 프로젝트가 동작하면 유지한다. 새 도구를 도입할 때는 얻는 기능, 설치·통합 비용, 남은 시간, 실패 시 대안을 함께 판단한다. 선택 이유는 "유명해서"보다 "기존 의존성을 사용해 핵심 검증 시간을 확보했다"처럼 실제 제약과 연결한다.

문서 구조

AGENTS.md에는 항상 필요한 짧은 지침, 스킬에는 특정 상황의 절차, 참고 문서에는 상세 내용을 둔다. 전체 백과사전을 매 요청마다 읽히지 않는다. 이 저장소의 START_HERE.md와 스킬 선택표를 사용한다. 일반 구조 근거: AGENTS.md 문서, 스킬 문서.

4. 세 개의 병렬 개념

터미널을 여러 개 여는 것, 독립 프로세스를 병렬 실행하는 것, 에이전트에게 일을 나누는 것은 다르다.

방법	무엇이 나뉘나	잘 맞는 일	주요 비용
터미널 탭·분할	셸과 실행 중 프로세스	서버 유지 + 명령 실행	위치·PID·포트 혼동
Bash 백그라운드	명령 프로세스	독립 lint·단위 테스트	CPU/RAM, 종료 코드 수집
여러 Codex 세션	대화와 작업 상태	경로가 분리된 독립 작업	컨텍스트 전달·파일 충돌
서브에이전트	위임한 작업과 반환 결과	읽기 조사·리뷰·분리된 구현	토큰·통합·검증 비용

터미널을 늘려도 Codex 추론이 자동으로 분산되지 않는다. 여러 세션이 같은 폴더를 열면 같은 파일을 본다. 대화가 나뉜다고 파일이 격리되는 것도 아니다. 이것은 프로그램 내부의 멀티스레딩을 켜는 설정과도 다르다.

권장 출발 배치

T1: 주 Codex 세션. 요구사항, 구현, 통합을 담당한다.

T2: 개발 서버. 실제 포트를 기록하고 계속 켜 둔다.

T3: 테스트·파일 확인. 필요할 때만 추가한다.

이 배치는 준비 전략이며 필수 개수가 아니다. 작은 과제에서는 두 탭으로 충분할 수 있다.

병렬화 판단

병렬로 실행할 일이 서로의 출력 파일을 쓰는가? 같은 DB를 초기화하는가? 같은 lockfile을 바꾸는가? 그렇다면 먼저 순서를 정한다. 독립 읽기·리뷰는 병렬화하기 쉽지만 설치·마이그레이션·공유 설정 수정은 충돌 가능성이 크다.

서브에이전트는 일반적으로 추가 모델·도구 작업을 하므로 토큰 사용도 늘 수 있다. 명시적인 위임 요청과 제공 기능을 확인한다. 근거: OpenAI 서브에이전트 문서.

5. Bash 병렬 실행과 종료 코드

다음 예시는 해당 npm 스크립트가 실제 package.json에 있고 서로 독립적일 때만 쓴다. 개발 서버는 이 예시에 섞지 않는다.

```
mkdir -p .playbook
npm run lint > .playbook/lint.log 2>&1 &
lint_pid=$!
npm run test:unit > .playbook/unit.log 2>&1 &
unit_pid=$!

lint_rc=0
wait "$lint_pid" || lint_rc=$?
unit_rc=0
wait "$unit_pid" || unit_rc=$?
printf 'lint=%s unit=%s\n' "$lint_rc" "$unit_rc"
```

`&`는 명령을 백그라운드에서 실행한다. `$!`는 직전에 시작한 작업의 PID다. 각 PID를 기다리고 각각의 종료 코드를 수집해야 한쪽 실패를 놓치지 않는다. 마지막 명령 하나가 성공했다고 두 작업이 모두 통과한 것은 아니다.

결과를 읽는 순서

종료 코드를 확인하고 실패한 로그의 관련 구간을 읽는다. 종료 0이어도 테스트가 실제로 실행됐는지, 테스트 0개를 통과로 보고한 것은 아닌지 확인한다. 로그의 예상·실제 값이 주장을 뒷받침해야 한다.

준비 도구로 기록할 때

```
python3 /tmp/codex-hackathon-playbook/scripts/playbook.py run \
  --workspace . --id V-001 --requirement R-001 \
  --track src/calculate.py --track tests/test_calculate.py \
  --timeout 90 -- python3 -m unittest discover -s tests -v
```

위 파일과 명령은 예시다. 실제 경로로 바꾼다. run은 지정 명령을 그대로 실행하므로 허용된 로컬 명령인지 확인한다. 재검증은 새 ID를 사용하고, 비밀을 명령 인자나 로그에 넣지 않는다.

6. 서브에이전트에게 일을 나누기

사용할 수 있는 기능인지 먼저 확인한다. 현재 일반 Codex 문서는 명시적인 요청 또는 적용되는 지침에 따라 위임하며 `/subagents`로 스레드를 확인하는 흐름을 설명한다. 실전 메뉴와 허용 범위가 우선이다. 근거: 서브에이전트 문서.

좋은 위임 계약

항목	전달할 내용 예시
목표	R-002 입력 검증의 누락 발견
입력	README의 조건, API 계약, 관련 파일
수정 범위	읽기 전용 또는 tests/ 아래 특정 파일
제외 범위	package.json·lockfile·공통 타입 변경 금지
완료 조건	재현 가능한 사례와 실제 테스트 결과
반환 형식	발견→영향→파일→검증→미확인 사항

바로 쓸 요청

> 서브에이전트가 지원되면 독립 검토를 위임해줘. 한 명은 요구사항과 현재 구현을 읽기 전용으로 비교하고, 한 명은 지정한 테스트 파일에서 경계값 사례를 작성해줘. 주 에이전트는 구현과 공유 파일, 통합을 맡아줘. 각각의 허용 경로와 완료 조건을 전달하고 결과를 검증해줘.

통합 전 확인

서브에이전트의 "완료" 보고는 증거 파일과 diff를 확인할 출발점이다. 주 에이전트가 요구사항·호출 경로·테스트 결과를 대조한다. 두 작업이 각각 성공해도 합쳐진 결과가 성공한다고 가정하지 않는다.

독립적인 일이 없거나 과제가 작으면 나누지 않는다. 전체 구현을 여러 에이전트에게 중복 수행시킨 뒤 하나를 고르는 방식은 시간·토큰과 통합 비용을 늘릴 수 있다.

7. 파일 충돌과 통합

같은 폴더의 여러 세션과 에디터는 파일을 공유한다. 작업을 시작하기 전에 파일 소유권을 정한다.

담당	허용 범위	주의할 공유 부분
주 에이전트	핵심 구현·공통 타입·패키지·통합	전체 계약 일관성
검증 담당	지정 테스트 파일	테스트 데이터·초기화 충돌
읽기 리뷰	변경 없음	실제 발견만 반환

API 요청/응답, 데이터 키·단위, 오류 형식은 병렬 구현 전에 합의한다. 예를 들어 서버가 비율 0.25를 반환하는데 UI가 25로 해석하면 각각의 코드는 정상이어도 결과는 틀린다.

에디터 충돌

AI·터미널이 수정한 파일을 에디터에서 미저장 상태로 다시 저장하면 변경을 덮을 수 있다. 상세 가이드가 안내하는 외부 변경 불러오기/현재 내용 저장 선택 전에 양쪽 내용을 비교한다. 저장 실패·충돌 표시가 사라졌는지 확인한다.

Git worktree는 선택 사항

일반 개발에서는 worktree로 파일 작업 공간을 분리할 수 있다. 다만 실제 환경이 지원하고, 현재 기준 커밋과 통합 절차를 이해할 때만 쓴다. 별도 작업 폴더의 변경은 제출 폴더에 자동 반영되지 않는다. 초기 과제에는 경로 소유권만 나누는 편이 더 단순할 수 있다.

로컬 Git 기록과 원격 Git push는 다르다. 시험 중 외부 push를 수행하지 않는다. 준비 저장소의 원격 주소를 실제 작업 결과의 업로드 대상으로 쓰지 않는다.

통합의 완료 조건

선택한 변경이 실제 제출 폴더에 있고, 공유 계약이 맞으며, 합쳐진 핵심 사용자 흐름이 확인되어야 한다. 문서의 실행법과 한계도 최종 상태로 갱신한다.

8. 대기·오류·자원 문제

응답이 멈춘 듯 보일 때는 먼저 상태를 분류한다.

상태	의미	행동
승인 대기	사용자의 결정을 기다림	명령·대상을 읽고 한 번 응답
실행 중	명령 또는 모델이 작업 중	출력 관찰, 동일 요청 중복 금지
서버 유지	종료 없이 서비스 제공 중	서버 탭 유지, 다른 탭에서 작업
완료	요청 결과가 반환됨	파일·실행 결과·화면을 확인
반복 오류	환경·모델·코드 원인 가능	오류와 시각을 기록해 원인별 대응

Browser가 열리지 않을 때

서버 터미널의 오류, 프로세스 종료 여부, 실제 포트, 0.0.0.0 바인딩, 경로 순서로 확인한다. 준비 안내의 3000은 예시다. 본인의 서버만 다시 시작하고 관리·연결 프로세스를 종료하지 않는다. 외부 터널이나 호스팅으로 우회하지 않는다. 근거: 상세 가이드 3쪽.

자원이 부족할 때

플랫폼의 CPU/RAM 표시와 본인이 실행한 작업 수를 확인한다. 중복 설치·빌드·테스트·서버를 줄인다. `killall node`나 `pkill python`처럼 대상을 넓게 종료하지 않는다. 같은 오류를 근거 없이 재시도하는 대신 원인 후보를 나누고 한 가지씩 확인한다.

질문을 보낼 곳

업무 정의는 실무자 에이전트, 화면 사용법·제출 절차는 Cofa Helper, 접속·반복 오류·감독 장애는 문제 신고로 구분한다. 발생 시각, 단계, 직전 행동, 오류 문구를 준비한다. 키·토큰은 포함하지 않는다. 신고·재접속이 시간을 정지하거나 연장한다고 가정하지 않는다.

9. 컨텍스트와 인수인계

같은 목표의 작업은 현재 세션에서 이어가되, 긴 작업의 핵심 상태는 파일에 남긴다. 대화 전체를 매번 다시 읽히는 것보다 현재 결정·실패·다음 행동을 정확히 전달하는 편이 유용하다.

인수인계에 필요한 정보

- 현재 목표와 실제 마감, 우선할 필수조건.
- 구현 위치와 실행 명령.
- 채택한 대안과 이유, 함부로 바꾸면 안 되는 계약.
- 마지막 검증의 명령·결과·로그.
- 지금 재현되는 오류와 아직 확인하지 않은 범위.
- 실행 중인 서버의 터미널·포트.
- 바로 할 다음 행동 1~3개.

복구 요청 예시

> .playbook/handoff.md와 과제 README를 읽고 현재 파일·diff·필요한 짧은 검증으로 상태를 대조해 줘. 완료한 부분을 다시 만들지 말고 다음 행동부터 이어가줘. 기록과 실제 상태가 다르면 차이를 먼저 알려줘.

일반 CLI에서는 `codex resume`으로 이전 세션을 선택할 수 있다. `/compact`는 긴 대화의 컨텍스트를 정리하는 기능이다. 제공 환경에서 해당 기능이 없으면 파일 기반 인수인계를 사용한다. 근거: Codex CLI 문서, 개발자 명령.

토큰을 줄이는 작업 습관

전체 저장소·전체 로그보다 필요한 파일과 오류 구간을 읽는다. 초기 탐색으로 얻은 사실을 짧게 기록하고 반복 조사를 줄인다. 이미 통과한 검증은 관련 변경이나 새로운 의심이 있을 때 다시 실행한다. 중요한 판단과 어려운 오류에 시간을 쓰고, 단순 파일 확인은 짧게 수행한다. 특정 모델의 속도·비용 우위를 측정 없이 보장하지 않는다.

10. 실제 검증과 증거

완료 보고에는 "무엇이 바뀌었는가"와 함께 "무엇을 실행해 무엇을 보았는가"가 있어야 한다. 이 준비 도구는 기록을 돕지만 의미 검증을 대신하지 않는다.

주장	필요한 확인	이것만으로는 부족
빌드 가능	실제 빌드 명령과 성공 결과	파일이 생성됨
API 동작	입력별 상태·본문·오류 검증	HTTP 200 한 번
계산 정확	작은 검산 입력과 기대값 비교	보기 좋은 그래프
버튼 동작	사람이 입력·클릭·결과 확인	정적 화면 렌더링
저장 유지	요구된 새로고침/재시작 범위 테스트	메모리에 잠시 남음

기록 단위

R-001 요구사항 → 구현 파일 → V-001 검증 → 실제 결과 → 남은 한계.

AI가 실행한 테스트와 사람이 직접 확인한 내용을 나눠 기록한다. 사람이 아직 Browser에서 확인하지 않았다면 "직접 확인 대기"로 둔다. 검증할 수 없었던 내용은 이유와 위험을 적는다.

하네스의 역할

run은 명령·종료 코드·시간·출력 로그·추적 파일 해시를 저장한다. audit은 필수 파일, 실행 기록, 추적 파일·로그 변경을 대조한다. 코드가 바뀌었으면 새 검증 ID로 재실행한다. 추적 목록 밖의 파일·의존성 변경은 자동 탐지하지 못한다.

`RECORDED_CHECKS_OK`는 등록된 기록이 맞는다는 뜻이다. 명령이 실제 주장에 맞는 테스트인지, 수동 UI 확인이 필요한지, 요구사항이 빠졌는지는 사람이 판단한다. 암호학적 서명이나 변경 불가능한 감사 로그도 아니다.

11. 시간 배분과 제출

제공 안내상 과제는 최대 180분이고 후속 서술형은 30분이다. 준비 대기와 응답 시간도 전체 종료 시각의 영향을 받는다. 다음 배분은 180분을 온전히 쓸 수 있는 경우의 예시이며 실제 남은 시간으로 조정한다.

경과 시간 예시	목표	판단 기준
0~15분	요구사항·데이터·실행 확인	첫 핵심 흐름과 검증 기준 확정
15~75분	핵심 흐름 구현	입력부터 결과까지 실제 연결
75~120분	필수 기능·경계 처리	필수 누락 우선, 부가 기능 제한
120~150분	검증·수정·직접 확인	실패 재현과 관련 회귀 확인
150~170분	기록·실행법·한계 정리	문서와 구현 일치
170~180분	제출 절차 시작	저장·필수 파일·확정 범위 확인

상세 가이드는 작업 종료 10분 전 제출 시작을 권장한다. 늦게 시작해도 전체 종료 시각이 연장되지 않는다. 약 5분 대기는 보장된 고정값이 아니며 화면의 실제 상태를 따른다. 근거: 브리프 1쪽, 상세 가이드 3~5쪽.

제출 전에 본인이 확인

필수 파일의 정확한 경로, 저장 완료와 충돌 해결, 실제 실행법, 핵심 동작, 문서의 한계를 확인한다. 제출 미리보기는 숨김 파일·내부 도구 폴더 등을 생략할 수 있다. 목록에 보이는 것만 제출된다고 단정하지 않는다.

제출물 확정 후에는 파일을 수정할 수 없다. 후속 서술형으로 이동한 뒤 AI를 이용하지 않는다. 마지막 완료 화면에서 제출 ID·시각·파일 수·크기를 확인한다. Codex 요청 완료, 파일 확정, 모든 응시 완료는 서로 다른 시점이다.

12. 바로 쓰는 요청문과 출처

구현 중

> 지금 필수 요구사항 중 아직 확인되지 않은 것을 알려줘. 가장 중요한 것 하나를 골라 구현하고, 정상·경계·실패 사례로 검증해줘. 실제 결과와 미검증 범위를 짧게 기록해줘.

막혔을 때

> 기대 결과와 실제 오류를 구분하고 가장 짧은 재현을 만들어줘. 원인 후보를 구분할 관찰부터 하고, 한 번에 작은 변경만 해줘. 동일 재시도를 새 근거 없이 반복하지 마.

제출 전

> 실제 과제 지침과 현재 결과물을 대조해 필수 누락, 문서와 코드의 모순, 오래된 검증을 찾아줘. 내가 Browser에서 직접 확인할 순서를 알려줘. 제출 클릭은 내가 진행할게.

이 저장소의 자료

START_HERE.md는 상황별 스킬 선택표다. templates/는 요구사항·판단·협업·검증·한계·인수인계 양식이다. scripts/playbook.py는 로컬 초기화와 실행 기록 도구다. docs/prompts.md에는 더 많은 요청문이 있다.

근거와 추가 읽기

- OpenAI: Subagents
- OpenAI: Build skills
- OpenAI: AGENTS.md
- OpenAI: Developer commands
- OpenAI: Codex CLI
- 대회 공개 홈페이지
- 사용자 제공 PDF 3개 및 추가 공지: 세부 시험 조건의 근거. 공개 홈페이지가 모든 세부 규정을 대신하지 않는다.

공개 스킬 6개 저장소의 검토 경로·커밋·선정 판단은 저장소 sources/catalog.json과 docs/source-selection.md에 있다. 이 가이드의 운영 예시와 시간 배분은 자체 준비안이며 공식 출제·채점 기준이 아니다.